

# 游戏客户端开发实习面试速记

Unity / C# 方向 · 口述精简版

|     |                                       |
|-----|---------------------------------------|
| 必须会 | 项目、FSM、事件中心、对象池、生命周期、协程、GC、Dictionary |
| 高频  | Unity性能、C++基础、TCP/UDP、进程线程            |
| 手写  | 链表、哈希、二叉树、BFS/DFS、二分                  |

## 01 项目与简历追问

Q 请用两分钟介绍 AUTO\_BABY\_CARE。

这是一个基于 Unity 独立开发的 3D 模拟经营原型。玩家需要在规定时间内同时照顾宝宝和经营商店，通过宝宝反馈、商品销售和每日评分形成核心循环。技术上，我使用分层有限状态机管理玩家、宝宝和顾客行为；使用事件中心解耦 UI、金钱和状态通知；使用对象池复用顾客、商品和提示 UI；使用 ScriptableObject 管理商品配置；并通过 UnityWebRequest 与协程接入 LLM。项目最大的收获是把多个玩法模块组织成可扩展、可维护的系统。如果继续迭代，我会重点完善存档、异常处理和性能测试。

Q 顾客 FSM 怎么实现？

StateMachine 保存当前状态，并提供 ChangeState。所有状态实现 IState 的 Enter、Update、Exit。切换时先执行旧状态 Exit，再替换当前状态并执行新状态 Enter。顾客按照进店、选择商品、移动、排队、结账、离店的流程切换。状态类通过顾客上下文读取数据并请求切换，但复杂项目可以把转换条件集中管理，减少状态之间的直接依赖。

Q 为什么使用 FSM？有什么缺点？

FSM 适合状态数量有限、转换规则明确的行为流程，能够避免大量 if-else，并把每种行为拆成独立模块。缺点是状态和转换增加后可能出现状态爆炸，状态之间也可能耦合。复杂决策可以考虑分层状态机、行为树或数据驱动转换表。

Q EventCenter 怎么实现？如何防止内存泄漏？

EventCenter 使用 Dictionary 保存事件标识与委托集合，订阅者注册回调，发布者只负责触发，从而降低业务与 UI 的直接依赖。全局事件中心会强引用订阅者，因此注册和注销必须成对：通常 OnEnable 注册、OnDisable 注销。还要防止重复注册，并在最后一个监听者移除后删除空事件；ClearAll 只能作为场景清理的兜底，不能替代正确注销。

Q 字符串事件名有什么问题？

字符串容易拼写错误、重构困难，而且缺少编译期类型检查。可以使用枚举、常量或泛型事件接口提升类型安全。事件中心也会隐藏调用关系，因此不是所有模块都应该使用事件，明确的一对一依赖可以直接调用。

Q 对象池怎么实现？

对象池按预制体或类型维护 Queue。取出时优先复用队列中的对象，没有可用对象时再实例化；回收时执行 OnDestroy、禁用对象并放回队列。OnSpawn 要重置位置、状态机、动画、刚体速度等；OnDespawn 要停止协程和 Tween、解绑事件、清理引用。对象池减少 Instantiate、Destroy 和临时内存分配，但低频或状态难以重置的对象不一定适合池化。

Q ScriptableObject 配置和运行时数据如何分离？

ScriptableObject 保存商品名称、价格、类型、图标等静态配置，原则上运行时只读。库存、购买数量、当前位置等动态状态放在普通运行时对象中。存档保存稳定 ID 和动态状态，加载后再通过 ID 找到配置，避免直接修改共享的 ScriptableObject 资源。

Q LLM 请求失败、格式错误和 Key 泄漏怎么办？

请求层需要处理超时、断网和非成功状态码，并提供固定文本兜底或有限次数重试。返回结果必须经过 JSON 解析、字段校验和规则层过滤，不能让模型直接修改权威游戏状态。正式项目不应把 API Key 写进客户端，应由业务服务器代为调用模型。对象销毁或场景切换时还应取消请求，防止过期回调更新已销毁对象。

Q AI 写了什么？如何证明你理解代码？

AI 主要用于提供备选方案、生成部分重复代码、辅助 Gizmos 和提供排查方向。需求拆分、架构选择、代码审查和验证由我负责。我能够解释合入代码的数据流、生命周期、设计取舍和潜在问题，也能独立写出 FSM、对象池和事件中心的核心逻辑。面试时最好补充一个真实案例：AI 方案哪里有问题、你如何定位、怎样修改和验证。

Q 项目只有一个月，为什么能完成这么多？

这是可玩原型，不是完整商业产品。我先完成宝宝照护和商店经营的最小闭环，再逐步增加 AI、UI 和表现层；部分资源和插件使用成熟方案，但核心业务脚本由我整合与维护。回答时应明确哪些功能完整、哪些仍是原型，不夸大完成度。

## 02 C# 核心八股

### Q 值类型和引用类型有什么区别？

值类型变量直接保存数据，赋值时通常复制数据；引用类型变量保存对象引用，赋值时复制引用，所以两个变量可能指向同一个对象。不能简单说值类型一定在栈：值类型作为对象字段时也会位于堆中。常见值类型有int、bool、enum、struct；常见引用类型有 class、string、数组和委托。

### Q 堆、栈和 GC 是什么？

栈用于保存方法调用产生的栈帧，方法结束后对应栈帧自动释放。引用类型对象通常分配在托管堆中；当对象已经无法通过任何有效引用访问时，GC 会在合适时机回收其托管内存。GC不是固定时间执行，频繁分配临时对象会增加 GC 压力并可能造成卡顿。UnityEngine.Object的引擎对象通常使用 Destroy，不能只依赖 GC。

### Q class 和 struct 的区别？

class 是引用类型，变量通常保存引用；struct 是值类型，赋值和传参通常复制整个值。struct适合较小、数据化、最好不可变的类型，过大的 struct 会增加复制成本。struct 也可能位于堆中，例如作为 class字段或数组元素。

### Q 接口和抽象类的区别？

接口描述能力与契约，一个类可以实现多个接口；抽象类既能定义抽象行为，也能保存字段和提供公共实现，但类只能继承一个基类。多个不相关对象需要统一能力时用接口；一组对象具有共同状态和基础实现时更适合抽象基类。

### Q 重载和重写有什么区别？

重载发生在同一个类型中：方法名相同、参数列表不同，编译期决定调用哪个方法。重写发生在继承体系中：子类用 override 提供 virtual 或 abstract 方法的新实现，运行时根据对象实际类型调用，体现多态。

### Q 委托和事件有什么区别？

委托是类型安全的函数引用，可以保存和调用一个或多个方法。事件是在委托基础上增加访问限制：外部通常只能订阅和取消订阅，不能直接触发或覆盖事件。全局事件长期持有实例方法时可能阻止订阅者被GC，因此必须正确解绑。

### Q 协程、线程和 async/await 的区别？

线程由操作系统调度，可以真正并行，但 Unity 大部分 API 只能在线程调用。Unity 协程通常仍在线程，通过yield 把流程拆到多个帧；协程中的重计算仍会卡主线程。async/await 是 C# 异步状态机语法，常用于网络 and 文件IO，本身不等于创建线程；Task.Run 才通常把计算放到线程池。

### Q Dictionary 的原理和复杂度？

Dictionary 基于哈希表。它先调用 GetHashCode 得到哈希值并定位桶，再沿桶关联的 Entry 比较哈希值和Equals。查询、添加、删除平均  $O(1)$ ，严重冲突时最坏可能退化为  $O(n)$ 。不同 Key落到同一桶称为哈希冲突，Dictionary 会通过 Entry 链继续查找；容量不足时会扩容并重建桶索引。

### Q List 的底层结构和复杂度？

List 底层是可扩容数组，元素连续存储。按下标访问  $O(1)$ ，尾部添加平均  $O(1)$ ；容量不足时需要申请更大的数组并复制旧元素，所以单次扩容是  $O(n)$ 。中间插入和删除需要移动后续元素，复杂度  $O(n)$ 。

### Q 什么是装箱和拆箱？

装箱是把值类型包装成 object或接口对象，通常会在托管堆产生分配；拆箱是从装箱对象中取出原值类型。频繁装箱会增加分配和 GC压力，泛型集合 List 通常比非泛型集合更能避免装箱。

### Q 深拷贝和浅拷贝？

浅拷贝只复制对象字段，其中引用字段仍指向同一子对象；深拷贝还会递归复制引用对象，使副本之间相互独立。深拷贝开销更大，应根据是否需要共享内部对象选择。

### Q 闭包为什么可能产生 GC？

Lambda捕获局部变量时，编译器通常需要生成一个额外对象保存被捕获状态，委托本身也可能产生分配。在高频 Update或循环中反复创建捕获式 Lambda，会增加 GC Alloc。可以缓存委托、减少捕获，或改为普通方法。

## 03 Unity 高频八股

### Q Unity 常见生命周期顺序？

通常是 Awake 初始化自身引用，OnEnable 在对象启用时执行，Start 在第一次 Update前且只执行一次。之后根据帧循环执行 FixedUpdate、Update、LateUpdate；禁用时执行OnDisable，销毁时执行 OnDestroy。不同对象之间的调用顺序默认不保证，注册与解绑最好成对放在 OnEnable和 OnDisable。

#### Q Update、FixedUpdate、LateUpdate 的区别？

Update 每渲染帧执行，适合输入和普通逻辑；FixedUpdate按固定物理步长执行，一帧可能执行零次或多次，适合 Rigidbody 物理操作；LateUpdate 在 Update后执行，适合相机跟随和依赖其他对象本帧更新结果的逻辑。移动速度通常乘 deltaTime，物理步长使用fixedDeltaTime。

#### Q OnDisable 和 OnDestroy 有什么区别？

对象或组件被禁用时会调用 OnDisable，之后重新启用还可能继续使用；OnDestroy在对象真正销毁时调用。若在 OnEnable 注册事件，应在 OnDisable解绑，避免禁用期间仍接收回调以及重复注册。永久资源释放可放在 OnDestroy，但不能只依赖 OnDestroy处理每次启停。

#### Q Collision 和 Trigger 的区别？

普通碰撞会参与物理响应并触发 OnCollision 系列回调；Trigger 不产生实体碰撞响应，只触发 OnTrigger系列回调，适合检测区域和拾取物。双方需要 Collider，通常至少一方需要 Rigidbody 才能获得预期的物理回调。

#### Q 为什么 Unity 大部分 API 只能在主线程调用？

场景、Transform 和渲染等引擎数据不是为任意线程并发访问设计的。工作线程可以处理不接触 Unity对象的纯计算或数据解析，完成后再把结果切回主线程更新场景。否则可能出现异常、数据竞争或不确定行为。

#### Q 怎么排查游戏卡顿？

先用 Profiler 判断瓶颈属于 CPU、GPU、渲染还是 GC，而不是直接猜。CPU 看 Timeline、脚本耗时和 GCAlloc；GPU 看 GPU Profiler、过度绘制和 Shader；渲染看Batches、SetPass、三角形数量；内存看纹理、网格和对象数量。定位到具体模块后再做对照测试。

#### Q 常见 CPU 和 GC 优化有哪些？

减少 Update 中的重复查找和临时分配；缓存组件引用；高频对象使用对象池；避免每帧字符串拼接、LINQ和捕获式Lambda；降低不必要的更新频率；大量实体可采用分帧更新、空间划分或数据导向方案。优化前后必须用Profiler 验证。

#### Q Draw Call 是什么？如何减少？

Draw Call 是 CPU 向 GPU 提交一次绘制命令。数量过多会增加 CPU提交和状态切换开销。可通过合批、共享材质、图集、GPUInstancing、合理的网格合并和减少材质变体来降低，但也要注意过度合并造成剔除变差和内存增加。

#### Q 静态合批、动态合批、GPU Instancing 的区别？

静态合批适合不移动物体，通常用额外内存换取较少的绘制提交；动态合批限制较多且有 CPU 合并成本；GPUInstancing 适合大量使用同一网格和材质的物体，通过一次或较少提交绘制多个实例。最终效果取决于渲染管线、材质和平台，应以 Profiler 为准。

#### Q UGUI 为什么会造成性能问题？

同一 Canvas 中元素发生布局、顶点或材质变化时，可能触发布局计算和 Canvas 重建。可以把高频变化与静态UI 拆到不同 Canvas，减少复杂 Layout Group 和频繁启停，使用图集并控制透明重叠。不要为了减少重建把所有UI 拆成大量 Canvas。

#### Q AssetBundle 和 Addressables 是什么？

AssetBundle 是 Unity 的资源打包与运行时加载机制，需要自己管理依赖、版本和卸载。Addressables 在其上提供地址、依赖、异步加载和远程更新等更高层能力。加载完成后要管理句柄与引用计数，错误释放会丢资源，不释放会造成内存泄漏。

## 04 C++ 基础

#### Q 指针和引用的区别？

指针保存地址，可以为空、重新指向其他对象并进行一定的地址操作；引用是已有对象的别名，声明时通常必须初始化，之后不能改成引用另一个对象。传参时，引用语法更直接；需要表达可空或重新绑定时更适合指针。

#### Q new/delete 和 malloc/free 的区别？

new 按类型分配内存并调用构造函数，delete 调用析构函数后释放；malloc只分配原始字节，不调用构造函数，free 只释放内存。两套方式必须配对，不能混用。现代 C++ 更推荐RAII、容器和智能指针，减少手动管理。

#### Q 什么是虚函数和多态？

基类把函数声明为 virtual 后，通过基类指针或引用调用时，会根据对象的实际类型选择子类实现，这叫运行时多态。常见实现通过虚函数表和对象中的虚表指针完成动态分派。用于多态删除的基类通常需要虚析构函数。

#### Q RAII 和智能指针？

RAII 把资源生命周期绑定到对象生命周期：构造时获得资源，析构时自动释放。unique\_ptr表示独占所有权；shared\_ptr使用引用计数共享所有权；weak\_ptr 不增加引用计数，常用于观察 shared\_ptr并打破循环引用。优先明确唯一所有权，避免滥用 shared\_ptr。

### Q vector 的原理和扩容？

vector 使用连续内存，按下标访问  $O(1)$ ，尾部添加平均  $O(1)$ 。size 表示元素数量，capacity 表示当前空间容量；容量不足时会申请更大内存并移动或复制元素。扩容可能使原来的指针、引用和迭代器失效，可以提前 reserve 降低重复扩容。

### Q map 和 unordered\_map 的区别？

map 通常基于平衡搜索树，键有序，查询、插入、删除  $O(\log n)$ ；unordered\_map 基于哈希表，键无序，平均  $O(1)$ ，最坏可能  $O(n)$ 。需要有序遍历和稳定对数复杂度时选 map，追求平均快速查找时常用 unordered\_map。

## 05 操作系统与网络

### Q 进程和线程的区别？

进程是资源分配和隔离的基本单位，拥有独立地址空间；线程是 CPU 调度的基本单位，同一进程内线程共享大部分资源。线程切换通常比进程轻，但共享数据需要同步，否则会发生数据竞争。

### Q 什么是死锁？产生条件是什么？

死锁是多个线程相互等待对方持有的资源，导致都无法继续。四个必要条件是互斥、占有且等待、不可抢占、循环等待。常见预防方法包括统一加锁顺序、减少锁范围、一次申请所需资源以及使用超时。

### Q TCP 和 UDP 的区别？

TCP 面向连接，提供可靠、有序、无重复的字节流，并包含重传和拥塞控制，但延迟与开销较高。UDP 无连接，不保证到达和顺序，开销低、时效性好。实时游戏可使用 UDP 并在业务层对关键消息增加序号、确认、重传或冗余；登录和支付等可靠业务常使用 TCP 或可靠协议。

### Q TCP 为什么三次握手？

客户端发送 SYN，服务端回复 SYN+ACK，客户端再回复 ACK。三次握手让双方确认彼此的发送和接收能力，并同步初始序列号；只进行两次无法让服务端确认客户端是否收到了自己的响应，也更容易受到历史失效连接的影响。

### Q TCP 为什么四次挥手？

TCP 是全双工连接，两端的发送方向需要分别关闭。一方发送 FIN 表示自己不再发送，另一方先 ACK，处理完剩余数据后再发送 FIN，最后由对方 ACK，因此通常表现为四次。主动关闭方会进入 TIME\_WAIT，确保最后 ACK 丢失时还能重发，并让旧报文在网络中消失。

### Q 帧同步和状态同步的区别？

帧同步主要同步玩家输入，各客户端执行相同逻辑得到相同结果，流量较小但要求逻辑确定性，断线重连和作弊处理更复杂。状态同步由权威服务器计算结果并同步状态，客户端负责预测、插值和表现，服务器压力与带宽更高，但权威性和容错通常更好。具体选择取决于玩法、人数、延迟和反作弊要求。

## 06 算法与现场手写

| 结构/算法   | 查询            | 插入/删除       | 要点            |
|---------|---------------|-------------|---------------|
| 数组      | 下标 $O(1)$     | 中间 $O(n)$   | 连续内存          |
| 链表      | $O(n)$        | 已知节点 $O(1)$ | 指针连接          |
| 哈希表     | 平均 $O(1)$     | 平均 $O(1)$   | 最坏 $O(n)$     |
| 平衡树     | $O(\log n)$   | $O(\log n)$ | 有序            |
| 堆       | 堆顶 $O(1)$     | $O(\log n)$ | TopK          |
| BFS/DFS | $O(V+E)$      | -           | 图与树遍历         |
| 常见排序    | $O(n \log n)$ | -           | 快排最坏 $O(n^2)$ |

必须能独立写：反转链表、链表判环、二叉树遍历、二分边界、两数之和、最长无重复子串、网格 BFS/DFS、简单对象池和 FSM。

写题顺序：复述题意与边界 → 说直接方案和优化方案 → 写代码并说明复杂度 → 测试空输入、单元素、重复元素和极端值。

面试底线：不知道就说明目前理解到哪里并给出排查思路；不要编造 API、性能数据或个人贡献。